

Correction - Complexité algorithmique

Term NSI — Octobre 2026

Algorithmes de recherche séquentiels et dichotomiques, complexité algorithmique, notation de Landau $O(g(n))$: $O(n)$, $O(\log n)$,...

1 Complexité des algorithmes : CORRIGÉ

1.1 Exercice 1 : Compter les opérations

1. On applique la règle des opérations significatives.

`multiplie1` :

Instruction	Coût
<code>L = []</code>	1
<code>L.append(b * i)</code> , exécutée n fois	n
<code>return L</code>	1

$$T(n) = n + 2$$

`multiplie2` : le corps de la boucle contient 2 opérations (`y = b * i` et `L.append(y)`), exécutées n fois.

$$T(n) = 1 + 2n + 1 = 2n + 2$$

2. Oui. Dans les deux cas, on garde le terme dominant et on supprime la constante multiplicative : $n + 2 \Rightarrow O(n)$ et $2n + 2 \Rightarrow O(n)$. Les deux fonctions sont **linéaires**. La différence vient d'un détail d'implémentation (une variable intermédiaire `y`), qui ne change pas la classe de complexité.

3. Le variant de boucle est `i` (plus rigoureusement `i + 1`, qui reste positif) :

- `i` vaut $n - 1$ au départ ;
- il diminue de 1 à chaque tour ;
- le dernier tour a lieu pour `i = 0`, puis `i` vaut -1 et la boucle s'arrête.

`i` prend les valeurs $n - 1, n - 2, \dots, 1, 0$: la boucle fait n itérations.

Le corps de boucle contient 2 opérations significatives (`y = b * i` et `L.append(y)`). Les lignes `i = n - 1`, `i >= 0` et `i = i - 1` servent à gérer le compteur et ne sont pas comptées.

$$T(n) = 1 + 2n + 1 = 2n + 2 \Rightarrow O(n)$$

4. Non : `multiplie2` (boucle `for`) et `multiplie3` (boucle `while`) ont exactement le même $T(n)$. La classe de complexité dépend de l'**algorithme**, pas du type de boucle utilisé.

1.2 Exercice 2 : Meilleur cas et pire cas

1. `est_present` :

- **meilleur cas** : 1 comparaison. `x` est le premier élément de la liste. Exemple : `est_present(3, [3, 8, 5, 1])`. La fonction s'arrête dès le premier `return True`.
- **pire cas** : n comparaisons. `x` est absent de la liste (ou en dernière position). Exemple : `est_present(7, [3, 8, 5, 1])`. Toute la liste est parcourue.

2. `compte` fait **toujours n comparaisons**, dans le meilleur comme dans le pire cas. Il n'y a pas de `return` dans la boucle : pour compter toutes les occurrences, il faut obligatoirement parcourir toute la liste.
3. Dans le pire cas, les deux fonctions font n comparaisons : elles sont toutes les deux en **O(n)**.
4. C'est `est_present` : la recherche séquentielle s'arrête dès que l'élément est trouvé. Dans une liste d'un million d'éléments, elle fait au maximum **1 000 000 comparaisons** (élément absent).

1.3 Exercice 3 : Déplacer le serpent dans Snake

1. Pour un serpent de taille n :

- la boucle fait n - 1 itérations, avec 1 opération chacune (`S[i] = S[i + 1]`) : n - 1 ;
- `S.pop()` : 1.

$$T(n) = (n - 1) + 1 = n$$

2. $T(n) = n \Rightarrow \mathbf{O(n)}$: complexité linéaire.

3. La complexité est linéaire : si la taille du serpent **double** (50 $\rightarrow$ 100 cases), le temps d'exécution de `supprime_queue` **double** aussi.

4. Non. `S.pop(0)` s'écrit en une seule instruction, mais Python doit en interne décaler tous les éléments suivants d'un cran vers la gauche, exactement comme notre boucle. C'est donc aussi une opération en **O(n)**. En revanche, `S.pop()` (suppression du dernier élément) ne déplace rien : elle est en O(1).

À retenir : une seule instruction Python n'est pas forcément une opération en O(1).

1.4 Exercice 4 : Boucles imbriquées

1. La boucle externe fait n tours. Pour chacun, la boucle interne fait n tours. `T.append(i * j)` est donc exécutée $n \times n = n^2$ fois.

$$T(n) = 1 + n^2 + 1 = n^2 + 2 \Rightarrow O(n^2)$$

2. Le pire cas est une liste **sans doublon** (toutes les valeurs sont différentes). La fonction ne rencontre jamais de `return True` et fait toutes les comparaisons possibles.

3. Pour n = 5 :

i	valeurs prises par j	nombre de comparaisons
0	1, 2, 3, 4	4
1	2, 3, 4	3
2	3, 4	2
3	4	1
4	aucune	0
	Total	10

4. Pour une liste de taille n, la boucle interne fait n - 1 tours pour i = 0, puis n - 2 tours pour i = 1, etc. :

$$T(n) = (n - 1) + (n - 2) + \dots + 2 + 1 + 0 = 1 + 2 + \dots + (n - 1)$$

La formule du cours, appliquée jusqu'à n - 1, donne :

$$T(n) = \frac{(n - 1) \times n}{2} = \frac{n^2}{2} - \frac{n}{2}$$

Vérification pour $n = 5$: $\frac{4 \times 5}{2} = 10$. On retrouve bien le total du tableau.

5. Le terme dominant est $\frac{n^2}{2}$; on supprime la constante $\frac{1}{2}$: $O(n^2)$.

`contient_doublon` fait environ **deux fois moins** d'opérations que `table` ($\frac{n^2}{2}$ contre n^2). Les deux fonctions sont pourtant dans la **même classe de complexité** : quadratique. Si n double, leur temps d'exécution est multiplié par 4 dans les deux cas.

1.5 Exercice 5 : Fonctions qui appellent des fonctions

1. La boucle de `truc` fait n tours avec 1 addition chacun :

$$T_{\text{truc}}(n) = n \Rightarrow O(n)$$

`truc(n)` ajoute n fois 1 à 0 : elle renvoie n .

2. D'après la question 1, l'appel `truc(i)` coûte i additions :

i	0	1	2	...	$n - 1$
additions dans <code>truc(i)</code>	0	1	2	...	$n - 1$

3. À chaque tour de la boucle de `chose`, on compte les additions de `truc(i)` **plus 1** addition (`res + ...`) :

$$T(n) = \underbrace{(0 + 1 + 2 + \dots + (n - 1))}_{\text{appels à truc}} + \underbrace{n}_{\text{additions de chose}}$$

$$T(n) = \frac{(n - 1) \times n}{2} + n = \frac{n^2}{2} + \frac{n}{2} = \frac{n(n + 1)}{2}$$

On garde le terme dominant $\frac{n^2}{2}$: $O(n^2)$.

4. `truc(i)` renvoie i , donc `chose(n)` calcule $0 + 1 + 2 + \dots + (n - 1) = \frac{n(n-1)}{2}$.

```
1 def chose_rapide(n):
2     return n * (n - 1) // 2
```

Cette fonction ne contient aucune boucle : elle fait toujours le même nombre d'opérations, quelle que soit la valeur de n . Elle est en $O(1)$.

À retenir : un peu de mathématiques peut faire passer d'un algorithme quadratique à un algorithme en temps constant.

1.6 Exercice 6 : Recherche dichotomique et logarithme

1. Cartes dans l'ordre 7, 8, 9, 10, J, Q, K, A :

```
1 L = [1, 2, 3, 4, 5, 6, 7, 8]
```

La Dame Q est représentée par 6 (indice 5 dans la liste).

2. Recherche de la Dame Q (valeur 6), avec `m = (g + d) // 2` :

g	d	m	L[m]	trouve
0	7	3	4	False
4	7	5	6	True

- 1^{re} itération : $L[3] = 4 < 6$, donc la Dame est à droite de m : $g = m + 1 = 4$.
- 2^e itération : $L[5] = 6$, la carte est trouvée.

La Dame est trouvée en **2 itérations**. Une recherche séquentielle aurait demandé **6 comparaisons** ($L[0]$ à $L[5]$).

3. Recherche de la valeur 9 (absente) :

g	d	m	L[m]	trouve
0	7	3	4	False
4	7	5	6	False
6	7	6	7	False
7	7	7	8	False
8	7			False

À chaque étape, $L[m] < 9$, donc $g = m + 1$. À la dernière ligne, $g > d$: la zone de recherche est vide et la boucle s'arrête.

La dichotomie fait **4 itérations**. La recherche séquentielle aurait fait **8 comparaisons** : il faut parcourir toute la liste pour conclure que la valeur est absente.

4.

- $8 \rightarrow 4 \rightarrow 2 \rightarrow 1$: **3 divisions**, et $\log_2(8) = 3$;
- $16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$: **4 divisions**, et $\log_2(16) = 4$;
- $32 \rightarrow 16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$: **5 divisions**, et $\log_2(32) = 5$.

Le nombre de divisions par 2 nécessaires pour passer de n à 1 est égal à $\log_2(n)$.

5. À chaque itération, la recherche dichotomique divise la zone de recherche par 2. Dans le pire cas, elle continue jusqu'à ce que la zone ne contienne plus qu'un élément (puis plus aucun). Le nombre d'itérations est donc environ $\log_2(n) + 1$: pour $n = 8$, on trouve bien $3 + 1 = 4$, comme à la question 3. On garde le terme dominant : la complexité est **$O(\log n)$** .

6. Nombre de comparaisons dans le pire cas (ordre de grandeur) :

Taille n de la liste	Recherche séquentielle	Recherche dichotomique
8	8	4
1 000	1 000	≈ 10
1 000 000	1 000 000	≈ 20
1 000 000 000	1 000 000 000	≈ 30

Remarque : $2^{10} = 1024 \approx 1000$. Multiplier n par 1000 n'ajoute donc qu'environ **10 étapes** à la recherche dichotomique, alors que la recherche séquentielle fait 1000 fois plus de comparaisons.

7. Il faut privilégier la **recherche dichotomique** : avec plusieurs centaines de milliers de mots, elle trouve un mot en une vingtaine d'étapes au plus, au lieu de plusieurs centaines de milliers. Elle n'est utilisable que si la liste de mots est **triée** (ici, par ordre alphabétique).

1.7 Exercice 7 : Défi

On compte les additions et les soustractions. Le `append` de `fonction_finale` n'est pas compté.

truc(n) : n additions.

$$T_{truc}(n) = n \Rightarrow O(n) \quad \text{renvoie } n$$

machin(n) : l'appel **truc(n)** coûte n additions, puis la boucle fait n soustractions.

$$T_{machin}(n) = n + n = 2n \Rightarrow O(n) \quad \text{renvoie } n - n = 0$$

chose(n) : à chaque tour i , l'appel **machin(i)** coûte $2i$ opérations, plus 1 addition (**res + ...**).

$$T_{chose}(n) = \sum_{i=0}^{n-1} (2i + 1) = 2 \times \frac{(n-1) \times n}{2} + n = n^2 - n + n = n^2$$

$$T_{chose}(n) = n^2 \Rightarrow O(n^2) \quad \text{renvoie } 0 \text{ (somme de zéros)}$$

fonction_finale(n) : la boucle fait n tours, et chacun appelle **chose(n)**, qui coûte n^2 opérations.

$$T(n) = n \times n^2 = n^3 \Rightarrow O(n^3)$$

Bonus : **chose(n)** renvoie toujours 0, donc **fonction_finale(n)** renvoie une liste de **n zéros**. Une version beaucoup plus efficace :

```
1  def fonction_finale_rapide(n):  
2      return [0] * n
```

Cette version est en **O(n)** : il faut bien créer les n cases de la liste. On passe d'un algorithme cubique à un algorithme linéaire. Pour $n = 1000$, cela représente environ 1 000 opérations au lieu d'un milliard.